

Математические основы алгоритмов, осень 2020 г.
Лекция 8: Суффиксное дерево, алгоритм Укконена. Сжатие
данных: метод Хаффмана, арифметическое кодирование*

Александр Охотин

4 апреля 2021 г.

Содержание

1 Суффиксные деревья (окончание)	1
1.1 Суффиксное дерево и его построение	1
1.2 Примеры использования суффиксного дерева	11
2 Сжатие данных	11
2.1 RLE	11
2.2 Код Хаффмана	11
2.3 Арифметическое кодирование	13

1 Суффиксные деревья (окончание)

1.1 Суффиксное дерево и его построение

Суффиксное дерево для w — это компактное префиксное дерево для множества суффиксов w . Дуги, как обычно, помечены парами позиций в строке, и из каждой внутренней вершины исходит не менее двух продолжений. При этом суффиксные ссылки сохраняются только между оставшимися внутренними вершинами. Пример — на рис. ??(снизу).

Число вершин в суффиксном дереве — $O(n)$, поскольку листьев не более чем $n + 1$.

В упрощённом суффиксном дереве всякая подстрока представлена вершиной. В настоящем же суффиксном дереве подстрока представляется тройкой из (1) вершины, (2) исходящей из неё дуги и (3) количества символов, отсчитанных по этой дуге. Например, в суффиксном дереве на рис. ??(снизу) нет отдельной вершины для подстроки abb , и она представляется тройкой (вершина a , дуга b , по ней 2 символа). Все операции над упрощённым суффиксным деревом можно реализовать на настоящем суффиксном дереве, работая с такими тройками вместо вершин.

Известно несколько алгоритмов построения суффиксного дерева для данной строки за линейное время от длины этой строки. Предлагаемый *алгоритм Укконена* [1995] на каждом шаге строит *неполное суффиксное дерево* для префикса $a_1 \dots a_i$, в котором не отражены

*Краткое содержание лекций, прочитанных студентам 1-го курса СПбГУ, обучающимся по программам «Математика» и «МАиАД», в осеннем семестре 2020–2021 учебного года. Страница курса: http://users.math-cs.spbu.ru/~okhotin/teaching/algorithms_2020/.



Рис. 1: Эско Укконен (род. 1950).

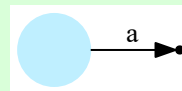
несколько последних суффиксов — а именно, *самый длинный суффикс, который присутствует в суффиксном дереве* (это значит, совпадает с какой-то ранее прочитанной подстрокой), и вместе с ним все более короткие суффиксы. При этом алгоритм будет помнить этот самый длинный суффикс, представленный в виде положения в дереве.

Положение в дереве — это пара (v, j) , где v — вершина дерева, а j , где $j \geq 0$ — длина, отсчитанная от этой вершины. Помнить, по какой именно дуге отсчитана вершина, алгоритму не нужно: это всегда будет переход по символу a_{i-j+1} , и по окончании обработки каждого символа a_i текущее положение всегда указывает на этот символ.

При чтении каждого очередного символа строки алгоритм пытается удлинить самый длинный суффикс на единицу. Если в дереве есть продолжение по прочитанному символу, то обработка символа этим и ограничится; если же такого символа в этом месте дерева нет, то дерево будет перестраиваться.

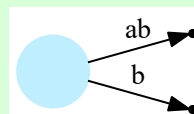
Пример 1 (Построение суффиксного дерева для $w = ababbabbba$). После чтения первого символа a в суффиксное дерево добавляется одна дуга, ведущая в лист. Недостроенных суффиксов нет; иными словами, текущая вершина — корень, смещение — 0.

Прочитано: **a**
Положение: корень, длина 0



После чтения второго символа b дуга a продлевается до дуги ab . Одновременно алгоритм вносит более короткий суффикс b в дерево. Недостроенных суффиксов нет.

Прочитано: **ab**
Положение: корень, длина 0

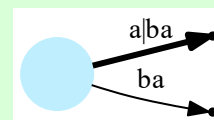


Чтобы продлить все суффиксы на один новый символ, для каждого листа необходимо приписать этот символ к строке на входящей дуге. Поскольку строки на дугах обозначены номерами позиций, нужно увеличить на единицу позицию конца строки. Так как листьев $O(n)$, уже одни эти нехитрые действия в совокупности займут квадратичное время. Поэтому используется следующее **улучшение**: дуги, входящие в листья, помечаются парами вида (i, ∞) , где значок бесконечности говорит о том, что дуга идёт до конца прочитанного префикса входной строки. После чтения очередного входного символа такие пары продлеваются сами собою, просматривать их не надо.

Продолжение примера 1. При чтении третьего символа a , обе идущие в листья дуги, которые помечены парами $(1, \infty)$ и $(2, \infty)$, сами собой продлеваются до aba и ba . Теперь суффикс a прочитанной строки aba совпадает с ранее прочитанной подстрокой — первым символом a . Алгоритм определяет это, пытаясь прочитать символ a из корня; эта попытка увенчивается успехом, и потому алгоритм запоминает суффикс a в виде тройки (корень, дуга по a , длина 1), и больше ничего не достраивает. Получается следующее неполное суффиксное дерево с одним недостроенным суффиксом, где пометка $a|ba$ означает длину 1, отсчитанную на дуге aba .

Прочитано: ***aba***

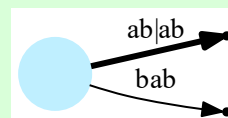
Положение: корень, дуга
a, длина 1



По прочтении четвёртого символа b дуги продлеваются до $abab$ и bab . Кроме того, алгоритм успешно прочитывает символ b из текущего положения, продвигаясь на одну позицию вперёд по дуге $abab$, по ней теперь отмечено два первых символа. Таким образом, алгоритм помнит недостроенный суффикс ab , встречавшийся ранее в виде подстроки.

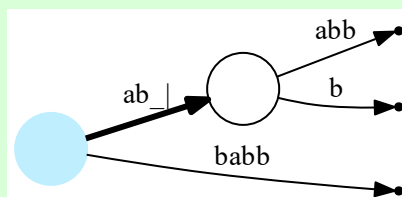
Прочитано: ***abab***

Положение: корень, дуга
a, длина 2



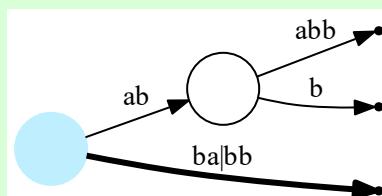
Затем читается пятый символ b и дуги продлеваются до $ababb$ и $babb$. Однако, прочитать символ b из текущего положения не удаётся: ведь следующий (третий) символ на дуге $ababb$ — это a . Поэтому алгоритм приступает к достраиванию суффиксов. Сперва дуга $ababb$ делится пополам промежуточной вершиной, остаётся дуга из корня в промежуточную вершину, помеченная подстрокой ab , а из промежуточной вершины исходит как старый кусок — abb , так и, отдельно, новый символ b .

Прочитано: **ababb**
 Положение: корень, дуга
a, длина 3
 (идёт обработка)



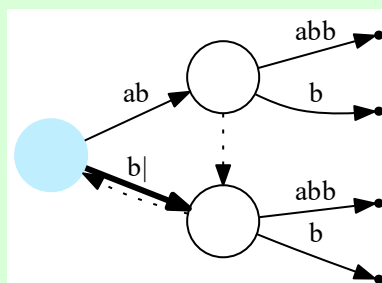
Суффикс *abb* таким образом достроен, однако следующий за ним суффикс *bb* остаётся недо-
 строенным — и алгоритм приступает к его достройке. Переход к следующему суффиксу
 происходит так: текущей вершиной ещё только что был корень, и на дуге *ababb* было от-
 мечено 3 символа; теперь надо отметить из корня на один символ меньше, начав читать
 оттуда суффикс *bb*.

Прочитано: **ababb**
 Положение: корень, дуга
b, длина 2
 (идёт обработка)



Вновь налицо несовпадение между последним отмеченным символом на дуге *babb* (*a*) и
 прочитанным пятым символом *b*. Дуга опять делится пополам промежуточной вершиной,
 соответствующей подстроке *b*, и ставится суффиксная ссылка из предыдущей промежу-
 точной вершины (*ab*) в эту (*b*); на рисунке она изображена пунктирной линией. Наконец,
 алгоритм переходит к чтению последнего суффикса *b*, отмечая из корня на один символ
 меньше. На этот раз отмеченный символ (*b*) совпадает с прочитанным пятым символом *b*, и
 потому производится заключительное действие — ставится суффиксная ссылка из послед-
 ней созданной вершины в текущую.

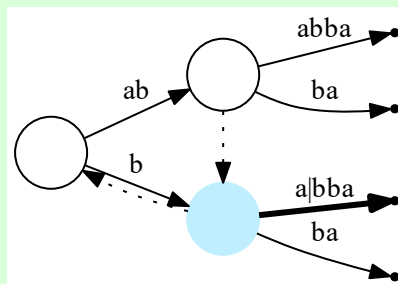
Прочитано: **ababb**
 Положение: корень, дуга
b, длина 1



После чтения шестого символа *a* дуги, ведущие в листья, продлятся до *abba* и т.д., а на
 выходящей из корня дуге *b* будет отмечена подстрока длины 2. Поскольку двух символов

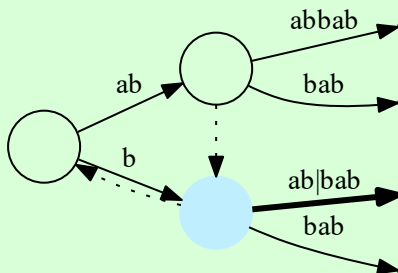
на этой дуге нет, алгоритм смещает текущую вершину в вершину b , откуда будет отмечена подстрока a длины 1. Подстрока совпадает с входным символом, поэтому никаких других изменений не потребуется.

Прочитано: ***ababba***
 Положение: вершина b ,
 дуга a , длина 1



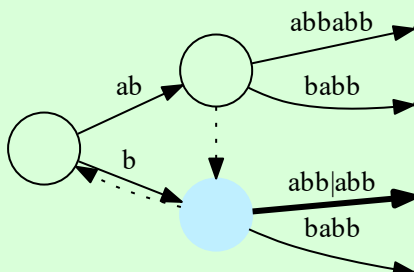
Следующий, седьмой символ b также успешно читается по той же дуге, больше ничего не делается.

Прочитано: ***ababbab***
 Положение: вершина b ,
 дуга a , длина 2



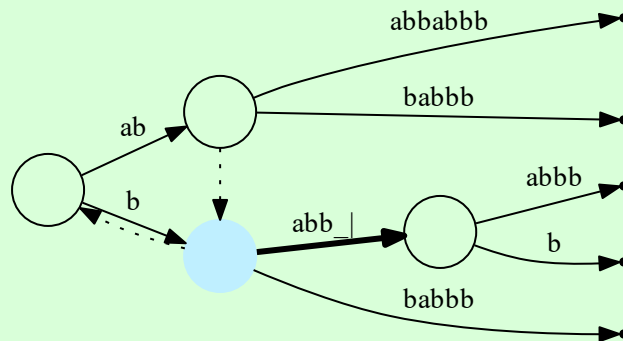
Восьмой символ b читается точно так же. К этому моменту накоплено уже четыре недостроенных суффикса.

Прочитано: ***ababbabb***
 Положение: вершина b ,
 дуга a , длина 3



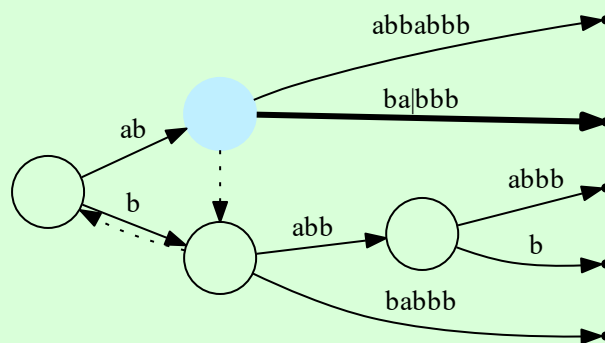
При чтении девятого символа b на текущей дуге отмечается четвёртый символ (a), и так как символы не совпадают, дуга разделяется.

Прочитано: **ababbabbb**
 Положение: вершина **b**,
 дуга **a**, длина 4
 (идёт обработка)

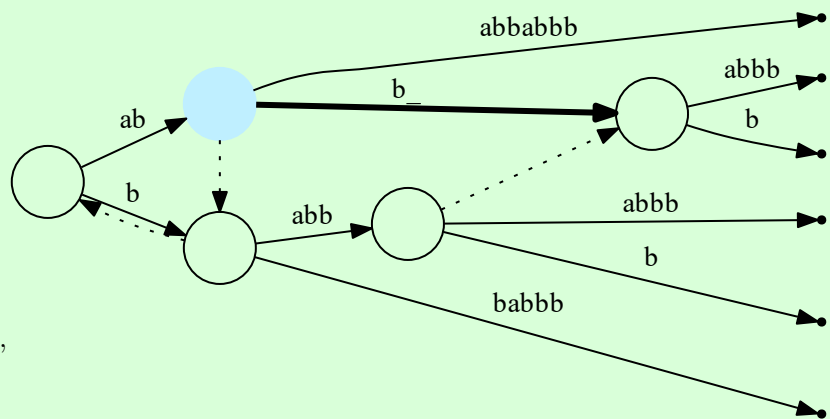


Далее алгоритм переходит к следующему суффиксу (**abbb**). Поскольку последний построенный суффикс (**babbb**) начинается во внутренней вершине (**b**) и находится на 4 позиции ниже, достаточно перейти из текущей вершины по суффиксной ссылке и отсчитать оттуда эти же 4 позиции. Суффиксная ссылка идёт в корень, и новое текущее положение — 4-я позиция по дуге **ab**. Поскольку на дуге **ab** четырёх символов нет, алгоритм переходит по ней к следующей вершине (**ab**), от которой отсчитываются два символа по дуге **b**.

Прочитано: **ababbabbb**
 Положение: вершина **ab**,
 дуга **b**, длина 2
 (идёт обработка)

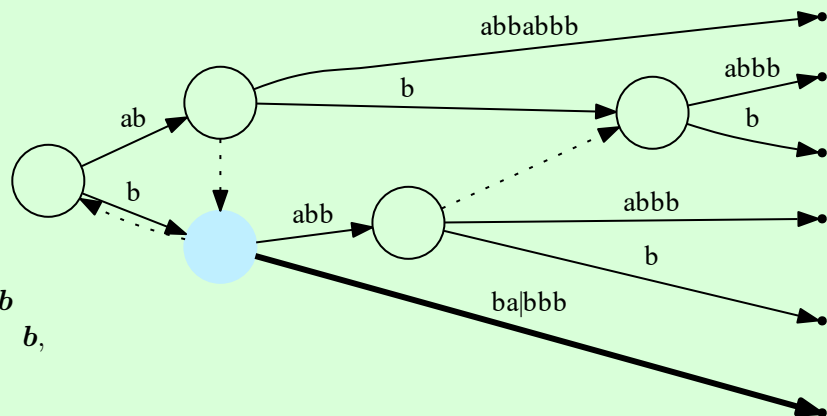


Здесь оказывается, что текущий символ на дуге (**a**) не совпадает с прочитанным (**b**), и потому дуга опять делится на две с созданием промежуточной вершины (**abb**). Из предыдущей промежуточной вершины (**babb**) сразу же ставится суффиксная ссылка на эту.



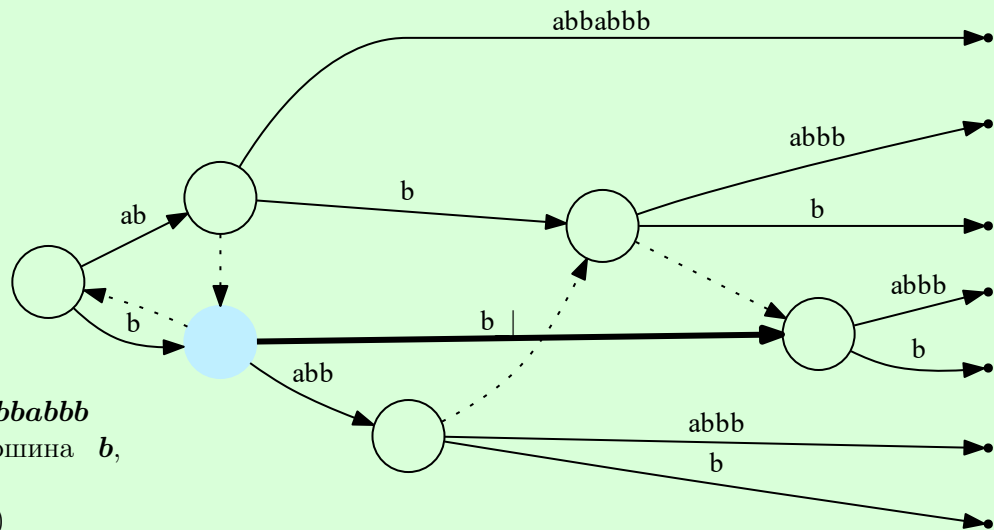
Прочитано: **ababbabbb**
 Положение: вершина **ab**,
 дуга **b**, длина 2
 (идёт обработка)

Таким образом, суффикс **abbb** внесён в дерево, алгоритм должен перейти к следующему суффиксу (**bbb**). Для этого достаточно перейти по суффиксной ссылке с сохранением количества позиций (2).



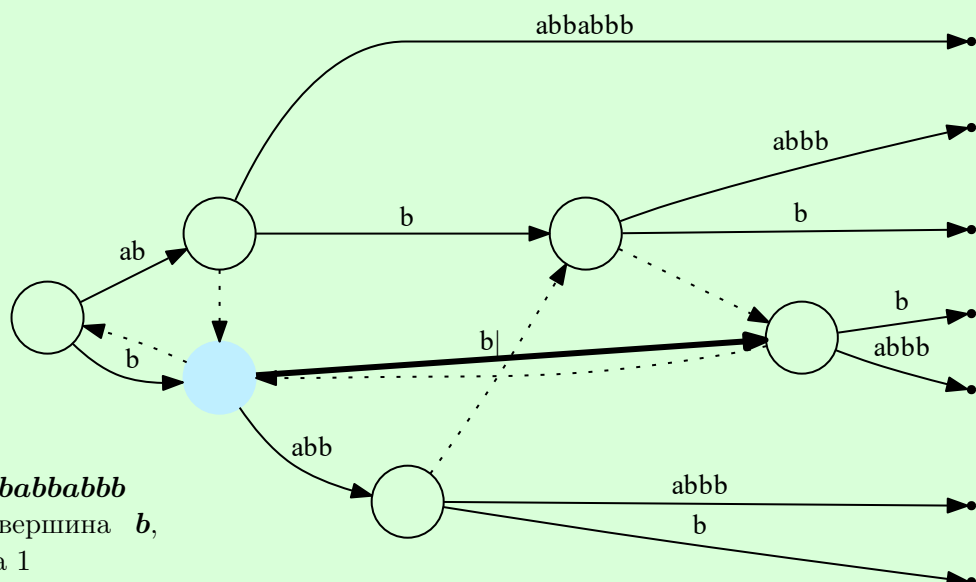
Прочитано: **ababbabbb**
 Положение: вершина **b**,
 дуга **b**, длина 2
 (идёт обработка)

Символ на дуге (**a**) вновь не совпадает с прочитанным символом (**b**), дуга делится на две, из предыдущей промежуточной вершины (**abb**) ставится ссылка на новосозданную (**bb**).



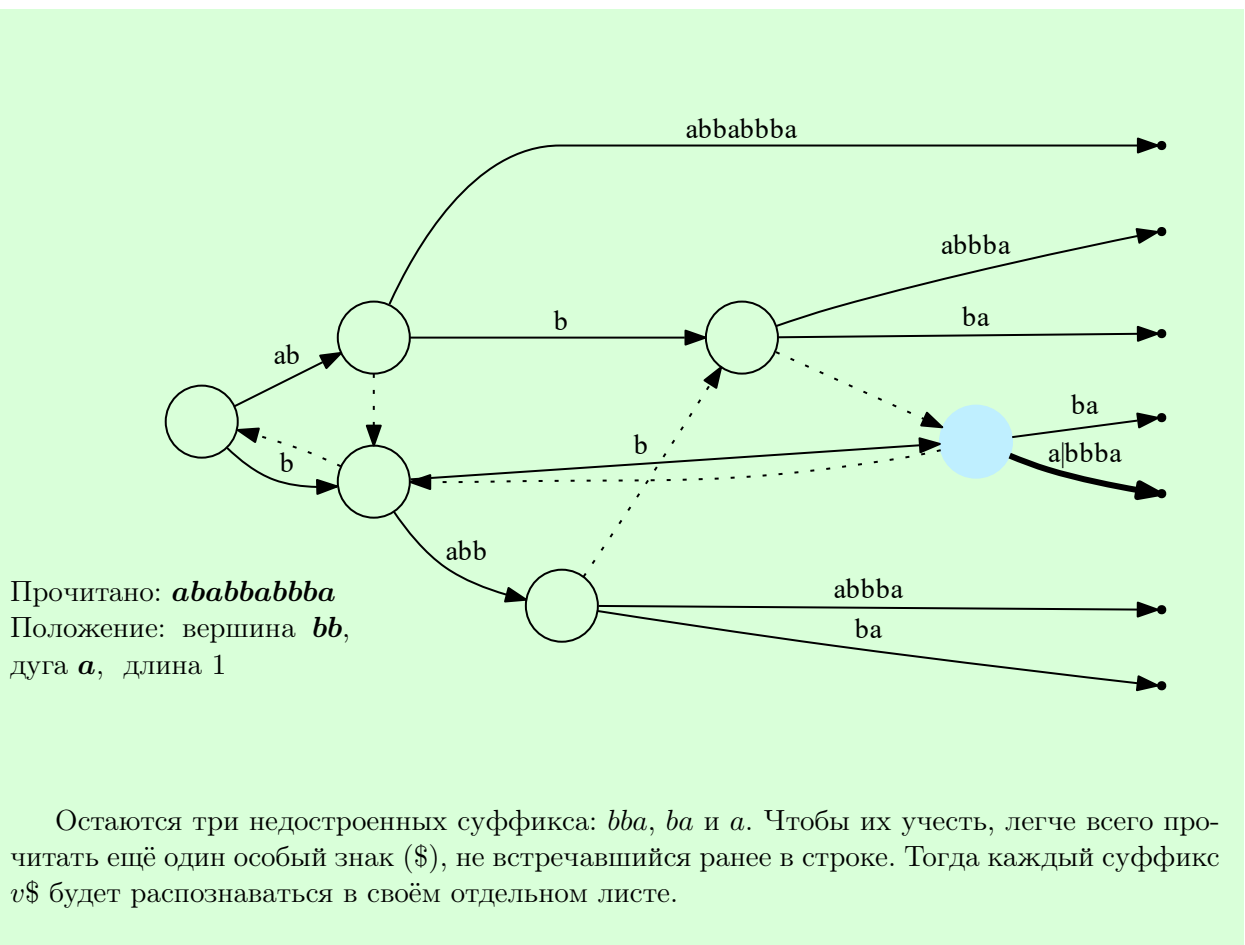
Прочитано: **ababbabbb**
 Положение: вершина **b**,
 дуга **b**, длина 2
 (идёт обработка)

Далее алгоритм переходит к следующему суффиксу (**bb**), следуя суффиксной ссылке с сохранением количества позиций (2) и попадая в корень, на дугу **b**. На дуге **b** нет двух позиций, поэтому алгоритм переходит по ней в вершину **b**. Здесь символ на дуге соответствует прочитанному, и алгоритму остаётся проставить суффиксную ссылку из последней созданной вершины (**bb**) в текущую (**b**).



Прочитано: **ababbabbb**
 Положение: вершина **b**,
 дуга **b**, длина 1

Читая последний символ **a**, алгоритм успешно продвигается вперёд, в вершину **bb**, на 1 позицию по дуге **a**.



Как всё это реализовать?

Когда следующий символ на текущей дуге — это как раз и есть входной символ, текущее положение продвигается на один символ вперёд, дерево не изменяется никак, и тем самым количество недостроенных суффиксов увеличивается на единицу. Если же на текущей дуге стоит какой-то другой символ, то её необходимо разрезать, создав в текущем положении новую промежуточную вершину, из которой выходят две дуги: остаток разрезанной дуги и новая дуга в ещё одну новую вершину, помеченная прочитанным символом.

Вставить символ в одном этом месте может быть недостаточно, нужно найти следующий суффикс. Если вершина в текущем положении — это корень, то следующий суффикс находится в одной из соседних дуг. Если же это не корень, то есть суффиксная ссылка (а суффиксные ссылки в суффиксном дереве такие же, как и в упрощённом суффиксном дереве), и следующий суффикс находится по ней.

На первый взгляд, алгоритм потенциально может работать квадратичное время — ведь цикл в строчке 6 может потребовать линейного числа итераций, да и вложенный в него цикл в строчке 7 сам по себе может долго работать. Тем не менее, время работы алгоритма всё-таки линейно.

Лемма 1. Алгоритм Укконена работает за время $O(n)$.

Доказательство. Значение j увеличивается всего n раз — в строчке 5. С другой стороны, каждая итерация цикла в строчке 6 уменьшает значение j в строчках 22–24, а каждая

Алгоритм 1 Алгоритм Укконена для построения суффиксного дерева

Вход: строка $w = a_1 \dots a_n$.

Переменные: выделенная вершина v ; смещение $j \geq 0$ относительно этой вершины; неявно заданная дуга (v, vx) , определяемая символом a_{i-j+1} .

```
1: записать дерево, состоящее из одного корня ( $\varepsilon$ ).
2:  $j = 0$ 
3:  $v = \varepsilon$ 
4: for  $i = 1$  to  $n$  do
5:    $j = j + 1$                                 /* попробовать продвинуться по дуге на один символ */
6:   while  $j > 0$  do
7:     while  $j > |x|$  do                          /* положение на дуге заходит за край */
8:        $j = j - |x|$ 
9:        $v = vx$                                     /* пройти по дуге */
10:    if из вершины  $v$  нет перехода по  $a_{i-j+1}$  then
11:      /* здесь всегда будет  $j = 1$  и  $a_{i-j+1} = a_i$ , раньше всё совпадает */
12:      добавить переход из  $v$  по  $a_i$  в новый лист  $va_i$ 
13:    else if  $j$ -й символ на дуге совпадает с  $a_i$  then
14:      if ранее (на  $i$ -й итерации) создавалась промежуточная вершина then
15:        добавить суффиксную ссылку оттуда в текущую
16:        перейти к следующей итерации по  $i$ 
17:      else
18:        разрезать дугу, вставить промежуточную вершину
19:        if ранее (на  $i$ -й итерации) создавалась промежуточная вершина then
20:          добавить суффиксную ссылку оттуда в новую
21:    if  $v$  — корень then
22:       $j = j - 1$ 
23:    else
24:       $v = \text{суффиксная\_ссылка}(v)$ 
```

итерация цикла в строчке 7 уменьшает j в строчке 8. Поэтому тело каждого из этих циклов будет выполняться не более чем n раз, и время работы всего алгоритма — $O(n)$. $\square$

Из линейного времени работы следует, что и само суффиксное дерево имеет линейный размер.

1.2 Примеры использования суффиксного дерева

Кроме поиска, суффиксное дерево позволяет выполнить другие интересные операции. «Вычислить количество вхождений подстроки x в w »: сперва спуститься из корня по строке x , а затем посчитать число листьев в поддереве. Действительно, каждое вхождение подстроки — это суффикс, начинающийся с этой подстроки, а каждому суффиксу в суффиксном дереве соответствует лист.

«Найти наибольшую общую подстроку w и x ». В своё время Дональд Кнут предположил, что эту задачу нельзя решить быстрее чем за время $n \log n$, где $n = |w| + |x|$. Но с помощью суффиксного дерева её нетрудно решить за линейное время. Делается это следующим обходом суффиксного дерева для w : сперва надо попробовать спуститься по x , и всякий раз, когда дальше спускаться будет некуда (то есть, целой подстроки x в w не содержится), надо подняться по суффиксной ссылке на уровень выше и продолжать дочитывать x . После прочтения всей подстроки x следует вернуть самую длинную из встреченных по дороге подстрок.

Почему это работает? Инвариант: если алгоритм пришёл в вершину u , это общая подстрока. Действительно, всякий переход по суффиксной ссылке из au сохраняет инвариант, сохраняет его и спуск по символу. Если общая подстрока v есть, то когда алгоритм прочитает её в x , он окажется в некоторой вершине uv суффиксного дерева.

Дополнительное чтение о суффиксных деревьях: Апостолико и др. [2016].

2 Сжатие данных

Задача: построить функцию $f: \Sigma^* \rightarrow \Sigma^*$, переводящую разные строки в разные, которая переводила бы практически встречающиеся строки в строки меньшей длины (перевести каждую строку в строку меньшей длины, очевидно, невозможно). Чаще всего, $\Sigma = \{0, 1, \dots, 255\}$.

2.1 RLE

Самый простой метод сжатия данных — использовать повторяющиеся последовательности одинаковых символов. Если символ повторяется два или более раз, то после второго повторения записывается число — сколько повторений ещё осталось.

Пример 2. $aaaaaabbabbb \rightarrow aa4bb4$; $abababab \rightarrow abababab$; $aabbaabb \rightarrow aa0bb0aa0bb0$.

Полезен, например, для сжатия изображений, содержащих большие одноцветные куски, и бесполезен для сжатия фотографий. Также оказывается полезен в качестве составной части более сложных алгоритмов.

2.2 Код Хаффмана

Пусть в строке какие-то символы встречаются чаще, какие-то реже. Это свойственно многим языкам; например, в рассказе Конан Дойля [1903] «Пляшущие человечки» Шерлок



Рис. 2: Дэвид Хаффман (1925–1999).

Холмс успешно расшифровывает сообщения, опираясь на различную частотность букв в английском языке. Это свойство можно использовать для сжатия данных.

Предлагается сопоставить каждому символу последовательность битов, и чем чаще этот символ встречается, тем короче будет последовательность. Эта же соображение использовано в азбуке Морзе, где коды букв фиксированы, и для чаще встречающихся букв предусмотрены более короткие коды («е»: точка; «ы»: тире-тире-точка-тире). Код Хаффмана — это по существу новая азбука Морзе, автоматически создаваемая для произвольной данной частотности символов.

Два алфавита, Σ и Ω . Гомоморфизм между моноидами Σ^* и Ω^* : условие $h(uv) = h(u)h(v)$. Гомоморфизм $h: \Sigma^* \rightarrow \Omega^*$ определяется образами всех односимвольных строк. Гомоморфизм — *код*, если прообраз однозначно восстанавливается по образу. *Префиксный код*: если код никакого символа не может быть префиксом кода другого символа: $h(a) \notin h(b)\Omega^*$; тогда по первым k символам $h(w)$, где k — константа, можно однозначно восстановить первый символ w .

Пример 3. $\Sigma = \{a, b, c\}$, $\Omega = \{0, 1\}$, $h(a) = 0$, $h(b) = 01$, $h(c) = 10$. Не код, поскольку $h(ac) = h(ba) = 010$.

Пример 4. $\Sigma = \{a, b, c\}$, $\Omega = \{0, 1\}$, $h(a) = 0$, $h(b) = 01$, $h(c) = 11$. Код, но не префиксный: $h(acccc) = 011111111$, $h(bccc) = 01111111$.

Пример 5. $\Sigma = \{a, b, c\}$, $\Omega = \{0, 1\}$, $h(a) = 0$, $h(b) = 10$, $h(c) = 11$. Префиксный код.

Код Хаффмана — это префиксный код, который строится для кодирования данной строки w в битовую строку — с тем, чтобы эта строка получилась как можно короче.

Число вхождений символа a в строку w обозначается через $|w|_a$. Для построения кода Хаффмана строится дерево. Сперва есть отдельные вершины, соответствующие символам, и помеченные числами $|w|_a$; они считаются поддеревьями. На каждом шаге берутся два поддерева с наименьшими значениями и соединяются в новое дерево с новым корнем, а два исходящих из корня ребра помечаются нулём и единицей. Если значения поддеревьев — m и n , то новому корню приписывается значение $m + n$. Так делается, все поддерева объединяются в одно дерево. Это — префиксное дерево кода Хаффмана.



Рис. 3: Йорма Риссанен (род. 1932).

При построении дерева Хаффмана удобно хранить поддеревья в очереди с приоритетами — тогда получится время $O(n \log n)$.

Теорема 1. Пусть Σ — алфавит, $w \in \Sigma^*$ — строка, $|w|_a$ — количество вхождений символа $a \in \Sigma$ в эту строку. Тогда код Хаффмана — это кратчайший префиксный код для w .

Доказательство. Индукция по размеру алфавита. Если $|\Sigma| = 2$, то один символ кодируется нулём, другой — единицей, и короче не бывает.

Переход. Пусть для всякого алфавита размера $|\Sigma| - 1$ и для всякого набора частот символов код Хаффмана оптимален. Берутся два символа, a и b , встречающихся в w реже всего. Заменив a и b в строке w на один символ $\hat{a}b$, встречающийся $|w|_a + |w|_b$ раз, по предположению индукции получается, что изменённая строка w' оптимально кодируется кодом Хаффмана h' . Код Хаффмана h для w будет содержать по одному лишнему биту для каждого a и b , то есть, $|h(w)| = |h'(w')| + |w|_a + |w|_b$.

Возвращаясь к алфавиту Σ , пусть g — произвольный префиксный код для w . Сперва g переделывается в код $\tilde{g}$ так, чтобы образы a и b отличались только в последнем бите — $\tilde{g}(a) = x0$ и $\tilde{g}(b) = x1$ — а длина кода не увеличилась бы: $|\tilde{g}(w)| \leq |g(w)|$. Действительно, в дереве g есть самая глубокая вершина с двумя потомками (вершины с одним потомком можно удалить, получив более краткий код). Если эти два потомка — это не a и b , то это символы, встречающиеся не реже, и потому их можно поменять местами с a и b , не ухудшив код.

Далее, для алфавита, в котором a и b объединены, по коду $\tilde{g}$ строится новый код g' , переводящий $\hat{a}b$ в x , а все остальные символы так же, как $\tilde{g}$. Тогда $|g(w)| \geq |\tilde{g}(w)| = |g'(w')| + |w|_a + |w|_b$. Поскольку код g' не может быть оптимальнее кода Хаффмана h' , верно неравенство $|g'(w')| \geq |h'(w')|$.

Получена следующая цепочка неравенств.

$$|g(w)| \geq |g'(w')| + |w|_a + |w|_b \geq |h'(w')| + |w|_a + |w|_b = |h(w)|$$

Неравенство $|g(w)| \geq |h(w)|$ означает, что код Хаффмана не хуже произвольного префиксного кода g . $\square$

2.3 Арифметическое кодирование

Пусть $\Sigma = \{a, b\}$, и a встречается вдвое чаще, чем b . Метод Хаффмана всё равно назначит им коды одинаковой длины — однобитные — и, таким образом, ничего не сожмёт. Как

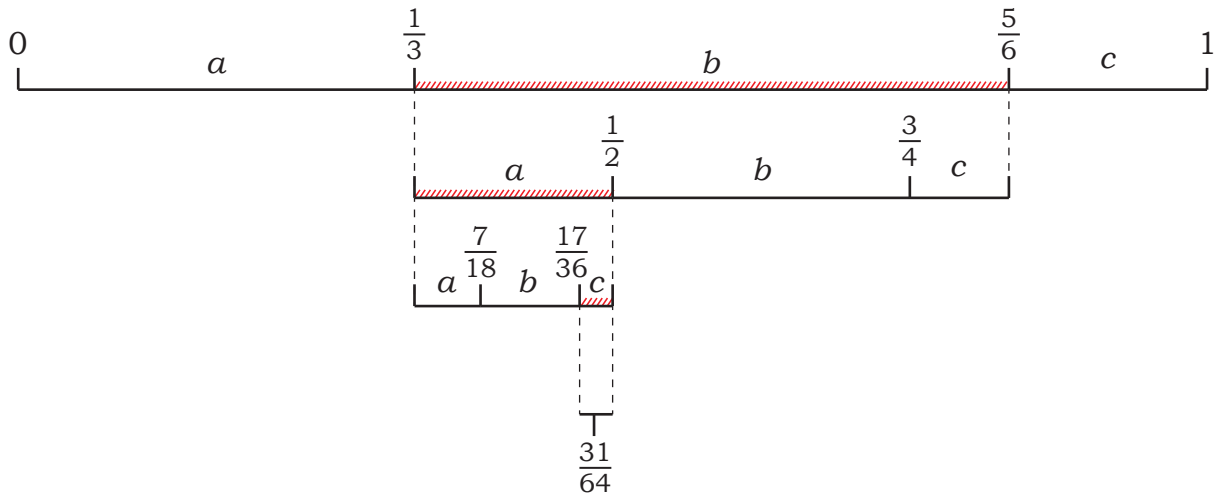


Рис. 4: Пример работы арифметического кодирования, строка $w = bac$ (пример 6).

воспользоваться их разной частотностью? *Арифметическое кодирование* (Риссанен [1976]) позволяет кодировать символы в каком-то смысле дробным числом битов.

Код всего сообщения — это одно рациональное число, лежащее в интервале $[0, 1)$. В каждый момент времени алгоритм помнит некоторый отрезок $[\ell, m)$, где ℓ и m — рациональные числа. Читая очередной символ входной строки, алгоритм переходит к отрезку меньшей длины, включённому в $[\ell, m)$.

Пусть алфавит — $\Sigma = \{a_1, \dots, a_k\}$, и символы имеют вероятности $P(a_1), \dots, P(a_k) \in \mathbb{Q}$, где $\sum_j P(a_j) = 1$. Если текущий интервал — $[\ell, m)$, и читается очередной символ, a_j , то интервал $[\ell, m)$ разбивается на кусочки, пропорциональные $P(a_1), \dots, P(a_k)$, и выбирается кусочек, соответствующий $P(a_j)$. Общая формула: от $[\ell, m)$ по символу a_j к $[\ell', m')$:

$$\begin{aligned}\ell' &= \ell + (m - \ell + 1)(P(a_1) + \dots + P(a_{j-1})) \\ m' &= \ell + (m - \ell + 1)(P(a_1) + \dots + P(a_{j-1}) + P(a_j))\end{aligned}$$

По завершению чтения всей строки выводится произвольное число из интервала $[\ell, m)$, а также количество символов. Зная это, а также зная исходный набор вероятностей, можно воспроизвести все шаги алгоритма кодирования и восстановить исходную строку

Пример 6. $P(a) = \frac{1}{3}$, $P(b) = \frac{1}{2}$, $P(c) = \frac{1}{6}$.

При чтении строки bac : $[0, 1) \rightarrow [\frac{1}{3}, \frac{5}{6}) \rightarrow [\frac{1}{3}, \frac{1}{2}) \rightarrow [\frac{17}{36}, \frac{1}{2})$. Тогда можно выдать число $\frac{31}{64} = (0.011111)_2$ — то есть, код 011111. Дополнительно запоминается, что всего символов три.

Как его раскодировать? Восстанавливается та же последовательность интервалов, одновременно выводятся символы строки. В начале интервал — $[0, 1)$, число $\frac{31}{64}$ попадает в интервал $[\frac{1}{3}, \frac{5}{6})$ — стало быть, первый символ — b , и т.д.

Как это реализовать? В виде чисел с плавающей точкой (float) — безнадёжно, их точность ограничена. Можно реализовать точную арифметику над рациональными числами, но это будет медленно: потребуется порядка n операций над числами с порядка n разрядами, что неизбежно займёт квадратичное время. Известна эффективная реализация, хранящая в памяти целые числа ограниченного размера. Старшие биты кода уже выведены и забыты, и алгоритм хранит разряды из середины. При этом веса символов воспроизводятся

немного неточно — например, с точностью до $\frac{1}{2^{32}}$ — но это ухудшает степень сжатия совсем незначительно.

Алгоритм будет работать с рациональными числами от 0 до 1, с шагом $\frac{1}{N}$, где N — степень двойки. Число $\frac{i}{N} \in \mathbb{Q}$ будет храниться в памяти в качестве целого числа i . Тогда начальный отрезок — $[\ell, m) = [0, N)$. Формула преобразования — такая же (алгоритм 2, строчка 6), однако вся арифметика становится целочисленной, и вычисленные значения округляются вниз. Если в какой-то момент ℓ и m принимают значения ниже $\frac{N}{2}$, это значит, что получаемая на выходе двоичная дробь имеет вид $(0.0\dots)_2$; поэтому первый ноль после запятой выводится, и обе границы диапазона умножаются на два. Если оба — не менее чем $\frac{N}{2}$, то дробь имеет вид $(0.1\dots)_2$, выводится 1, из обеих границ вычитается $\frac{N}{2}$, после чего обе границы умножаются на два.

Есть определённое затруднение: если ℓ будет приближаться к $\frac{N}{2}$ снизу, а m — сверху, то алгоритм не будет знать следующий двоичный разряд, однако точность будет стремительно теряться. Что делать? Когда ℓ будет больше $\frac{N}{4}$, и одновременно m — меньше $\frac{3N}{4}$, обе границы будут раздвигаться вдвое, а поскольку алгоритм не будет знать, какой бит при этом выводить — это зависит от того, по какую сторону от $\frac{N}{2}$ они в итоге окажутся — он будет лишь запоминать, сколько раз границы были раздвинуты. По какую сторону от $\frac{N}{2}$ они окажутся, станет ясно при выводе следующего бита — тогда и можно будет вывести все запомненные биты (строчки 15 и 20).

Наконец, в момент окончания входной строки нужно выдать последние биты, которые попадут в текущий интервал. Если $\ell < \frac{N}{4}$, то тогда $m > \frac{N}{2}$ (иначе алгоритм дальше раздвигал бы границы), и можно, с учётом запомненных битов, вывести число, меньшее чем $\frac{N}{2}$ (строчка 23); если же $\ell \geq \frac{N}{4}$, то тогда известно, что $m \geq \frac{3N}{4}$, и потому выдаётся число, большее чем $\frac{N}{2}$ (строчка 23).

При обратном преобразовании алгоритм хранит три рациональных числа от 0 до 1, точно так же закодированных с шагом $\frac{1}{N}$. Это те же два числа ℓ и m , а также лежащее между ними число x , представляющее значение закодированной последовательности — точнее, его биты, рассматриваемые в данный момент. Изначально в число x загружаются $\log N$ первых битов закодированной последовательности. Далее, на каждом шаге диапазон от ℓ до m разбивается пропорционально частотности символов, и проверяется, в область, соответствующую какому символу, попадает x . При каждом раздвигании ℓ и m вдвое, число x масштабируется вместе с ними — это значит, что старший разряд x выталкивается из буфера — и одновременно с этим во младший разряд x загружается следующий непрочитанный бит закодированной последовательности.

Замечание. Алгоритм не требует, чтобы набор вероятностей был одним и тем же на каждом шаге. Он может быть произвольным — главное, чтобы при кодировании и при декодировании он вычислялся одинаково. Можно изобретать статистические модели, оценивающие вероятность каждого символа в данном контексте, и выводить их через арифметическое кодирование.

Список литературы

- [2016] A. Apostolico, M. Crochemore, M. Farach-Colton, Z. Galil, S. Muthukrishnan, “40 years of suffix trees”, *Communications of the ACM*, 59:4 (2016), 66–73.
- [1903] A. Conan Doyle, “The return of Sherlock Holmes: The adventure of the dancing men”, *The Strand Magazine*, 26:156 (1903), 603–617.
- [1952] D. A. Huffman, “A method for the construction of minimum-redundancy codes” *Proceedings of the IRE*, 40:9 (1952), 1098–1101.

Алгоритм 2 Арифметическое кодирование

Дано: строка $w = a_{i_1} \dots a_{i_n}$ над алфавитом $\Sigma = \{a_1, \dots, a_k\}$, вероятности символов $P(a_1), \dots, P(a_{j+1})$ — целые числа, сумма которых равна N .

```
1:  $\ell = 0$ 
2:  $m = N$ 
3: запомнено_битов = 0
4: for  $i = 1$  to  $n$  do
5:   Пусть  $a_j$  —  $i$ -й символ  $w$ 
6:    $(\ell, m) = (\ell + \frac{m-\ell}{N} \cdot (P(a_1) + \dots + P(a_{j-1})), \ell + \frac{m-\ell}{N} \cdot (P(a_1) + \dots + P(a_j)))$ 
7:   while что-то можно сделать do
8:     if  $\ell \geq \frac{N}{4}$  и  $m < \frac{3N}{4}$  then                                /*  $\ell = 0.01x, m = 0.10y$  */
9:        $\ell = 2(\ell - \frac{N}{4})$                                           /*  $\ell \leftarrow 0.0x$  */
10:       $m = 2(m - \frac{N}{4})$                                            /*  $m \leftarrow 0.1y$  */
11:      запомнено_битов = запомнено_битов + 1
12:     else if  $m < \frac{N}{2}$  then                                         /*  $\ell = 0.0x, m = 0.0y$  */
13:        $\ell = 2\ell$                                                     /*  $\ell \leftarrow 0.x$  */
14:        $m = 2m$                                                         /*  $m \leftarrow 0.y$  */
15:       Вывести  $01^{\text{запомнено\_битов}}$ 
16:       запомнено_битов = 0
17:     else if  $\ell \geq \frac{N}{2}$  then                                         /*  $\ell = 0.1x, m = 0.1y$  */
18:        $\ell = 2(\ell - \frac{N}{2})$                                           /*  $\ell \leftarrow 0.x$  */
19:        $m = 2(m - \frac{N}{2})$                                            /*  $m \leftarrow 0.y$  */
20:       Вывести  $10^{\text{запомнено\_битов}}$ 
21:       запомнено_битов = 0
22:   if  $\ell < \frac{N}{4}$  then
23:     Вывести  $01^{\text{запомнено\_битов}}$ 
24:   else
25:     Вывести  $10^{\text{запомнено\_битов}}$ 
```

Алгоритм 3 Арифметическое кодирование: обратное преобразование

Дано: последовательность битов, её длина n , алфавит $\Sigma = \{a_1, \dots, a_k\}$, вероятности символов $P(a_1), \dots, P(a_{j+1})$ — целые числа, сумма которых равна N .

По сравнению с алгоритмом 2, обратное преобразование хранит текущее значение — число x , всегда лежащее между m и ℓ .

```
1:  $\ell = 0$ 
2:  $m = N$ 
3:  $x = (\text{первые } \log_2 N \text{ битов входа})$ 
4: for  $i = 1$  to  $n$  do
5:    $d = m - \ell$ 
6:   Пусть  $a_j$  — такой символ, что  $\frac{m-\ell}{N} \cdot (P(a_1) + \dots + P(a_{j-1})) \leq x < \frac{m-\ell}{N} \cdot (P(a_1) + \dots + P(a_j))$ 
7:   Вывести символ  $a_j$ 
8:    $(\ell, m) = (\ell + \frac{m-\ell}{N} \cdot (P(a_1) + \dots + P(a_{j-1})), \ell + \frac{m-\ell}{N} \cdot (P(a_1) + \dots + P(a_j)))$ 
9:   while что-то можно сделать do
10:    if  $\ell \geq \frac{N}{4}$  и  $m < \frac{3N}{4}$  then
11:       $\ell = 2(\ell - \frac{N}{4})$ 
12:       $m = 2(m - \frac{N}{4})$ 
13:       $x = 2(x - \frac{N}{4})$ 
14:      if следующий бит входа — единица then
15:         $x = x + 1$ 
16:    else if  $m < \frac{N}{2}$  then
17:       $\ell = 2\ell$ 
18:       $m = 2m$ 
19:       $x = 2x$ 
20:      if следующий бит входа — единица then
21:         $x = x + 1$ 
22:    else if  $\ell \geq \frac{N}{2}$  then
23:       $\ell = 2(\ell - \frac{N}{2})$ 
24:       $m = 2(m - \frac{N}{2})$ 
25:       $x = 2(x - \frac{N}{2})$ 
26:      if следующий бит входа — единица then
27:         $x = x + 1$ 
```

- [1976] J. J. Rissanen, “Generalized Kraft inequality and arithmetic coding”, *IBM Journal of Research and Development*, 20:3 (1976), 198–203.
- [1995] E. Ukkonen, “On-line construction of suffix trees”, *Algorithmica*, 14:3 (1995), 249–260.